

ZYVORAI LABS · CUSTOMER DOCUMENTATION

Zyvor Argus

Admin Basics

2026-09-02

Ports & URLs

Port	Surface	Notes
30080	Mission Control (lab / k3s NodePort)	Persisted by <code>deploy-remote.sh</code>
8080	Local <code>argus serve --port 8080</code>	Dev default
443 / TLS	<code>argus serve --tls</code>	When terminating TLS in-process
8090	Argus Enterprise (Watchfloor)	Customer package / Helm Service
30180	Bundled Keycloak NodePort (typical lab)	Only when Enterprise uses NodePort SSO

```
curl -s http://127.0.0.1:8080/health
```

```
remote lab example:
```

```
curl -s http://<host>:30080/health
```

Auth — Mission Control (community)

- When `DASHBOARD_PASSWORD` is set, `/dashboard` and `/api/dashboard/*` require a session from `/login`.
- Lab defaults are often `admin / Admin@321` — rotate for anything exposed.
- Override with `DASHBOARD_USER / DASHBOARD_PASSWORD` or host file `.argus-auth`.
- `GET /health` stays open for probes.

Auth — Argus Enterprise (SSO / OIDC)

Argus Enterprise (Watchfloor) does **not** use `DASHB0ARD_PASSWORD` . It uses OIDC (bundled Keycloak or your IdP) and optionally local username/password.

Demo accounts after a default Keycloak install (`keycloak.createTestUsers=true`):

Username	Password	Notes
demo	demo	Quick manual login
ssouser	Sso@321	Default for <code>scripts/test-login.sh</code>

These are smoke-test only — turn them off for production. Full install options (Helm, BYO OIDC, local auth, manual Docker, troubleshooting): [Enterprise SSO / OIDC](#).

```
scripts/test-login.sh https://argus.example.com demo demo
```

Key environment

Variable	Purpose
ZYVOR_BASE_URL	Target under test
LLM_PROVIDER / keys	OpenAI, Anthropic, Azure, Google, Ollama
ZYVOR_PRODUCT_REPO	GitHub owner/repo for specs / issues
ENABLE_MULTI_BROWSER	Firefox + WebKit projects
ENABLE_AUTH_SETUP	Playwright setup → storageState
ENABLE_EMULATION_PROJECTS	Dark / reduced-motion / locale projects
ZYVOR_GREP / ZYVOR_SHARD	Default suite filters
ZYVOR_HAR_PATH	Default HAR for replay
ENABLE_REGRESSION / VISUAL_MAX_DIFF_RATIO	Visual pipeline threshold
ENABLE_AUTOFIX / ENABLE_AUTOFIX_APPLY	Self-healing suggestions / apply
GITHUB_WEBHOOK_SECRET	HMAC for POST /webhook/github

Canonical list: repo .env.example and docs/configuration.md .

Deploy sketch

```
/scripts/deploy-remote.sh <host> <user> --quick --service # systemd needs free port  
/scripts/deploy-remote.sh <host> <user> --k3s # preferred when :30080 is NodePort
```

If k3s already binds **:30080** as the `argus-webhook` NodePort, prefer `--k3s` — bare systemd cannot steal that port.

Logs: `journalctl -u argus -f` or `kubectl logs deploy/argus-webhook` .

For Argus Enterprise Helm / Keycloak deploy steps, see [Enterprise SSO / OIDC](#).

Security notes

- Always set `DASHBOARD_PASSWORD` before exposing Mission Control pod logs publicly.
- For Enterprise, disable Keycloak demo users (`demo` / `ssouser`) outside eval labs.
- Treat webhook secrets, OIDC client secrets, and LLM keys as production secrets.
- Support: [GitHub issues](#) · sales@zyvor.dev for Enterprise.

Related

- [Getting Started](#)
- [Install prerequisites](#)
- [Using the Dashboard](#)
- [Enterprise SSO / OIDC](#)
- [Login \(Mission Control\)](#)

Operate from the console (UX)

1. Open this route from the nav or command palette and wait for live API data.
 2. Use filters/search when present; drill into a row for detail.
 3. For mutating actions: confirm role gates and impact before applying.
 4. **Empty / fail:** Check service health, auth, and that required CRDs/backends for this domain are installed.
 5. **Success:** Live data loads; created/updated objects appear without error toasts.
-

How to sign into **Argus Enterprise** (Watchfloor) after install: bundled Keycloak, your own OIDC provider, or a local username/password for a test lab.

This chapter is for the **Argus Enterprise** customer package (Helm chart + scripts). Community Mission Control (`argus serve`) still uses `DASHBOARD_PASSWORD` — see [Admin basics](#).

Demo accounts (smoke-test only)

After a default install with bundled Keycloak and test users enabled:

Username	Password	Purpose
demo	demo	Quick manual login check
ssouser	Sso@321	Scripted / <code>test-login.sh</code> smoke test

These are not production accounts. Disable them for a real customer environment

(`keycloak.createTestUsers: false` in Helm, or delete the users in Keycloak after you verify login).

```
scripts/test-login.sh https://argus.example.com demo demo
```

```
u
```

```
scripts/test-login.sh https://argus.example.com ssouser 'Sso@321'
```

Option A — Helm with bundled Keycloak (recommended)

```
# Load the image from the customer package first (see that package's INSTALL.md), then
helm install argus charts/argus-enterprise-*.tgz \
  --set appUrl=https://argus.example.com \
  --set keycloak.externalUrl=https://sso.example.com \
  --set keycloak.ingress.enabled=true \
  --set keycloak.ingress.className=<your-ingress-class> \
  --set keycloak.ingress.host=sso.example.com
```

What this does:

- Deploys the app, Postgres, and Keycloak.
- A post-install Job creates the `argus` realm, the `argus-enterprise` OIDC client, and (by default) the `demo` / `ssouser` test users above.
- Secrets (Postgres password, Keycloak admin password, OIDC client secret) are generated once and kept stable across `helm upgrade`.

Required values (no safe default):

- `appUrl` — browser-reachable URL of the app
- `keycloak.externalUrl` — browser-reachable URL of Keycloak

Keycloak must be reachable from browsers via **Ingress** (`keycloak.ingress.*`) or a **NodePort** (`keycloak.nodePort`). The chart fails install if neither is set.

After pods are ready:

1. Read the one-time owner setup token from app logs: `kubectl logs -n <ns> deploy/argus | grep -A3 -i "one-time"`
2. Open `appUrl`, claim ownership with your email + that token.
3. On the login screen, pick the SSO / Keycloak provider and sign in with `demo` / `demo` (or `ssouser` / `Sso@321`) to confirm OIDC.

Turn off test users for production:

```
helm upgrade argus charts/argus-enterprise-*.tgz \
  --reuse-values \
  --set keycloak.createTestUsers=false
```

(Also delete any already-created `demo` / `ssouser` users in the Keycloak admin console if they remain.)

Option B — Bring your own OIDC provider

Okta, Azure AD, Google Workspace, an existing Keycloak, or any standards- compliant OIDC IdP:

```
helm install argus charts/argus-enterprise-*.tgz \
--set appUrl=https://argus.example.com \
--set keycloak.enabled=false \
--set oidc.issuer=https://login.example.com/realms/your-realm \
--set oidc.clientId=argus-enterprise \
--set oidc.clientSecret=<secret>
```

Register this redirect URI on your IdP:

```
https://argus.example.com/api/enterprise/auth/callback/default
```

Demo Keycloak users do **not** apply here — use accounts from your IdP.

Option C — Local username/password (test lab, no IdP)

When you have nowhere to run Keycloak and no external IdP:

```
helm install argus charts/argus-enterprise-*.tgz \
  --set appUrl=https://argus.example.com \
  --set keycloak.enabled=false \
  --set localAuth.enabled=true
```

1. Claim the deployment with email + setup token **and** a password (the claim form shows a password field when local auth is on).
2. That password becomes the owner's login.
3. Owners can add more local accounts later via `POST /admin/local-accounts` .

Local auth and SSO can run together: the login screen shows both when both are configured. Prefer real SSO for any production deployment.

Option D — Manual / plain Docker (no Helm)

From the Enterprise package:

1. **Deploy Keycloak** (skip if you already have an OIDC provider):

```
kubectl create namespace argus-enterprise \
kubectl create secret generic argus-keycloak-admin -n argus-enterprise \
--from-literal=password="$(openssl rand -hex 20)" \
kubectl apply -f k8s/keycloak-theme-configmap.yaml \
kubectl apply -f k8s/sso-keycloak.yaml
```

2. **Create a client secret** and provision realm + test users:

```
CLIENT_SECRET="$(openssl rand -hex 32)" \
KEYCLOAK_ADMIN_PASSWORD="<password from step 1>" \
scripts/keycloak-bootstrap.sh <k8s-host> <ssh-user> "$CLIENT_SECRET" http://<app-host>:8090
```

This creates `ssouser / Sso@321` and `demo / demo`. Re-run after any Keycloak pod restart if you use the ephemeral `start-dev` manifest (`k8s/sso-keycloak.yaml`) — a restart wipes the realm.

3. **Deploy the app** wired to that client:

```
DEPLOY_OIDC_ISSUER="http://<keycloak-host>:30180/realms/argus" \
DEPLOY_OIDC_CLIENT_ID="argus-enterprise" \
DEPLOY_OIDC_CLIENT_SECRET="$CLIENT_SECRET" \
DEPLOY_EXTRA_ENV="ARGUS_ENTERPRISE_DB_URL=postgres://user:pass@host/db,ARGUS_ENTERPRISE_DB_USER=user" \
scripts/deploy-remote.sh <app-host> <ssh-user>
```

4. **Claim** via the one-time token in `docker logs`, then log in with `demo / demo`.

Keep the same `$CLIENT_SECRET` for Keycloak and the app. If they drift, login fails with a generic OAuth error — re-run bootstrap with the same secret.

For local auth only (no Keycloak):

```
DEPLOY_EXTRA_ENV=ARGUS_ENTERPRISE_LOCAL_AUTH_ENABLED=true \
scripts/deploy-remote.sh <app-host> <ssh-user>
```

Troubleshooting

Symptom	Likely cause	Fix
SSO button missing	No IdP registered / OIDC env not set	Check Helm <code>oidc.*</code> or <code>DEPLOY_OIDC_*</code> ; claim as owner and register an IdP if using the admin API
Login loops / generic OAuth error	Client secret mismatch	Re-run <code>keycloak-bootstrap.sh</code> with the same secret the app has
SSO worked, then broke after Keycloak restart	Ephemeral Keycloak storage wiped the realm	Re-run <code>keycloak-bootstrap.sh</code> (or let the Helm post-upgrade Job re-provision)
User created but can't finish login	Keycloak <code>VERIFY_PROFILE</code> wants first/last name	Use <code>keycloak-bootstrap.sh</code> / the Helm Job (they set names); or fill profile in Keycloak
Need a login with no IdP	Local auth off	<code>--set localAuth.enabled=true</code> or <code>ARGUS_ENTERPRISE_LOCAL_AUTH_ENABLED=true</code>

Related

- [Which product](#) — Community vs Watchfloor
- [Install prerequisites](#) — Docker/Helm/Community/token first
- [Admin basics](#) — Mission Control `DASHBOARD_PASSWORD` auth
- [Getting Started](#)
- [Login \(Mission Control\)](#)
- Engineering mirror (same content, repo docs tree): `../enterprise-sso-oidc.md`
- Sales / managed deploys: [zyvor.dev](#) · sales@zyvor.dev

Operate from the console (UX)

1. Open this route from the nav or command palette and wait for live API data.
2. Use filters/search when present; drill into a row for detail.
3. For mutating actions: confirm role gates and impact before applying.
4. **Empty / fail:** Check service health, auth, and that required CRDs/backends for this domain are installed.
5. **Success:** Live data loads; created/updated objects appear without error toasts.